
NAME : TREVOR MUSIIWA
SRN : 31232103
BSC STATISTICS – DATA SCIENCE
VISHWAKARMA UNIVERSITY

EMOTION DETECTION SYSTEM

Test Design, Test Scenarios & Test Cases

1. Introduction

This document defines the test design, test scenarios, and test cases for the Emotion Detection System a transformer-based NLP application that classifies text into seven emotion categories: joy, sadness, anger, fear, disgust, shame, and guilt. The document covers all testable components of the system including data preprocessing, model inference, sarcasm detection, negation handling, custom tokenization, batch processing, and the Streamlit web application frontend.

Field	Details
Project Name	Emotion Detection from Text Using Transformer-Based Models
System Under Test	Emotion Detection Pipeline + Streamlit Web Application
Document Type	Test Design, Test Scenarios & Test Cases
Version	1.0
Testing Approach	Functional, Integration, Edge Case, UI Testing

2. Test Design

2.1 Test Objectives

The primary objectives of testing the Emotion Detection System are:

- Verify that the data preprocessing pipeline correctly cleans, normalizes, and encodes text data from the ISEAR dataset.
- Validate that the DistilBERT model correctly classifies text into the correct emotion category after fine-tuning.
- Confirm that the custom tokenizer correctly expands abbreviations and slang before tokenization.
- Ensure that sarcasm detection correctly identifies sarcastic text and adjusts the predicted emotion.
- Verify that double negation handling correctly resolves constructions such as 'not bad' and 'not unhappy'.
- Validate that batch prediction processes multiple inputs correctly and produces downloadable output.
- Confirm that the Streamlit web application loads, accepts input, and displays results correctly.
- Ensure that the prediction logging system correctly records all predictions with timestamps.

2.2 Test Scope

Module	In Scope	Out of Scope
Data Collection	ISEAR loading, CSV saving, label verification	Twitter/Reddit API (not used)
Data Preprocessing	Cleaning, normalization, encoding, balancing, splitting	Manual label correction
Data Augmentation	EDA techniques, augmented file output	Back-translation (optional)
Model Architecture	DistilBERT loading, forward pass, output shape	Model architecture modification
Model Training	Loss convergence, checkpoint saving, training curves	Hyperparameter search
Custom Tokenizer	Abbreviation expansion, encoding correctness	Adding new abbreviations
Sarcasm Detection	Rule-based detection, emotion flip logic	Social media API integration
Negation Handling	Double/single negation detection, flip logic	Complex multi-clause negations
Inference Pipeline	Single prediction, confidence scores, all scores	Multilingual inference
Batch Prediction	CSV upload, bulk prediction, CSV download	Excel/JSON input formats
Streamlit App	All 4 tabs, model loading, UI interactions	Mobile/tablet UI testing
Logging	Prediction log CSV output, timestamps	Log rotation, log server

2.3 Test Approach

Testing is conducted using a combination of the following approaches:

- Black-box functional testing: Inputs are provided and outputs verified against expected results without inspecting internal code logic.
- White-box unit testing: Individual functions such as `clean_text()`, `normalize_text()`, and `expand_abbreviations()` are tested in isolation.
- Integration testing: The end-to-end pipeline from raw text input to final emotion prediction is tested as a whole.
- Edge case testing: Boundary inputs including empty strings, very long text, sarcastic expressions, double negations, and pure slang are tested.
- UI testing: The Streamlit web application is tested manually for correct rendering, user interactions, and output display.

2.4 Test Environment

Component	Specification
Operating System	Windows 10/11
Python Version	Python 3.12
Key Libraries	PyTorch 2.x (CPU), Transformers 4.x, Streamlit 1.x, Pandas, Scikit-learn
Hardware	CPU only (no GPU required)
Browser	Google Chrome / Microsoft Edge (for Streamlit UI testing)
Dataset	ISEAR (7,503 samples, 7 emotion classes)
Model	DistilBERT fine-tuned on ISEAR

2.5 Entry and Exit Criteria

Entry Criteria

- All Python dependencies are installed successfully.
- ISEAR dataset has been downloaded and saved to `data/raw/isear.csv`.
- The model has been trained and `best_model.pt` exists in the `models/` directory.
- `label_map.json` exists in `data/processed/`.
- Streamlit is installed and `step4_app.py` is present in the project directory.

Exit Criteria

- All High priority test cases pass.
 - At least 90% of Medium priority test cases pass.
 - All identified defects are documented with severity and status.
 - Test report is generated and reviewed.
-

3. Test Scenarios

The following high-level test scenarios cover all major functional areas of the Emotion Detection System.

Scenario ID	Scenario Description	Module	Priority
TS-01	Verify ISEAR dataset loads correctly with 7,503 samples and 7 emotion labels	Data Collection	High
TS-02	Verify text cleaning removes URLs, HTML tags, mentions, and special characters	Preprocessing	High
TS-03	Verify text normalization expands contractions and handles slang correctly	Preprocessing	High
TS-04	Verify label encoding maps all 7 emotions to correct integer values	Preprocessing	High
TS-05	Verify class balancing produces equal samples per emotion class	Preprocessing	Medium
TS-06	Verify dataset splits into 70/15/15 train/val/test with stratification	Preprocessing	High
TS-07	Verify EDA augmentation generates correct number of augmented samples	Augmentation	Medium
TS-08	Verify augmented samples retain their original emotion labels	Augmentation	High
TS-09	Verify DistilBERT model loads and produces correct output shape	Model Architecture	High
TS-10	Verify custom tokenizer correctly expands abbreviations before tokenization	Custom Tokenizer	High
TS-11	Verify trained model correctly predicts emotions for standard sentences	Model Inference	High
TS-12	Verify confidence scores are valid probabilities summing to 1.0	Model Inference	High
TS-13	Verify sarcasm detection identifies sarcastic sentences correctly	Sarcasm Detection	High
TS-14	Verify emotion is flipped correctly when sarcasm is detected	Sarcasm Detection	High
TS-15	Verify double negation is detected and emotion is not flipped	Negation Handling	High
TS-16	Verify single negation of positive emotion triggers emotion flip	Negation Handling	Medium
TS-17	Verify prediction on empty string is handled gracefully	Model Inference	High
TS-18	Verify prediction on very long text (>512 tokens) is handled by truncation	Model Inference	Medium

TS-19	Verify Streamlit app launches without errors	Streamlit UI	High
TS-20	Verify single prediction tab returns correct emotion with confidence	Streamlit UI	High
TS-21	Verify batch prediction processes CSV upload and returns results	Streamlit UI	High
TS-22	Verify prediction log records entries with timestamps correctly	Logging	Medium
TS-23	Verify A/B comparison tab loads and compares two models	Streamlit UI	Medium
TS-24	Verify model prediction on pure slang text after abbreviation expansion	Custom Tokenizer	Medium
TS-25	Verify system handles missing model file with a clear error message	Error Handling	High

4. Test Cases

Detailed test cases are provided below, organized by module. Each test case includes a unique ID, pre-conditions, step-by-step test steps, expected results, and fields for recording actual results and status.

4.1 Data Collection Module

Test Case ID	TC-DC-001
Title	ISEAR Dataset Load — Sample Count Verification
Module	Data Collection
Priority	High
Test Type	Functional
Pre-conditions	Python environment set up. data_collection.py is present. Internet connection available.
Test Data	N/A — automated download
Test Steps	1. Run: python data_collection.py 2. Observe terminal output for sample count.
Expected Result	Terminal prints '[DONE] Total samples collected: 7503'. File data/raw/isear.csv is created.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-DC-002
Title	ISEAR Dataset Load — Label Distribution Verification
Module	Data Collection
Priority	High
Test Type	Functional
Pre-conditions	data_collection.py has run successfully. isear.csv exists.
Test Data	N/A
Test Steps	1. Run: python data_collection.py 2. Observe label distribution in terminal output.
Expected Result	All 7 emotions present (anger, disgust, fear, guilt, joy, sadness, shame). Each class has approximately 1,000-1,100 samples.
Actual Result	
Status	Pass / Fail

Remarks	
Test Case ID	TC-DC-003
Title	ISEAR Dataset Load — Null Label Verification
Module	Data Collection
Priority	High
Test Type	Functional
Pre-conditions	data_collection.py has run successfully.
Test Data	N/A
Test Steps	1. Run: python data_collection.py 2. Check output line: 'Any nulls in label column'
Expected Result	Terminal prints 'Any nulls in label column: 0'
Actual Result	
Status	Pass / Fail
Remarks	

4.2 Data Preprocessing Module

Test Case ID	TC-PP-001
Title	Text Cleaning — URL Removal
Module	Data Preprocessing
Priority	High
Test Type	Unit
Pre-conditions	data_preprocessing.py functions available. Dependencies installed.
Test Data	Input: 'Check this link https://example.com for more info'
Test Steps	1. Call clean_text('Check this link https://example.com for more info') 2. Observe returned string.
Expected Result	Returned string: 'Check this link for more info' (URL removed, extra space may remain, collapsed by whitespace regex).
Actual Result	
Status	Pass / Fail
Remarks	
Test Case ID	TC-PP-002

Title	Text Cleaning — HTML Tag Removal
Module	Data Preprocessing
Priority	High
Test Type	Unit
Pre-conditions	data_preprocessing.py functions available.
Test Data	Input: ' I feel so angry today '
Test Steps	1. Call clean_text(' I feel so angry today ') 2. Observe returned string.
Expected Result	Returned string: 'I feel so angry today' (all HTML tags removed).
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-PP-003
Title	Text Normalization — Contraction Expansion
Module	Data Preprocessing
Priority	High
Test Type	Unit
Pre-conditions	data_preprocessing.py functions available. contractions library installed.
Test Data	Input: "I can't believe they didn't tell me"
Test Steps	1. Call normalize_text("I can't believe they didn't tell me") 2. Observe returned string.
Expected Result	Returned string contains 'cannot' and 'did not' (contractions expanded).
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-PP-004
Title	Text Normalization — Slang Expansion
Module	Data Preprocessing
Priority	High
Test Type	Unit
Pre-conditions	data_preprocessing.py functions available.
Test Data	Input: 'omg smh i cannot believe this rn'

Test Steps	1. Call <code>normalize_text('omg smh i cannot believe this rn')</code> 2. Observe returned string.
Expected Result	Returned string: 'oh my god shaking my head i cannot believe this right now'
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-PP-005
Title	Label Encoding — All 7 Emotions Mapped
Module	Data Preprocessing
Priority	High
Test Type	Functional
Pre-conditions	Preprocessed DataFrame with all 7 emotions available.
Test Data	ISEAR preprocessed DataFrame
Test Steps	1. Run <code>encode_labels(df)</code> 2. Check <code>label_map</code> keys and values.
Expected Result	<code>label_map</code> contains exactly 7 keys: anger, disgust, fear, guilt, joy, sadness, shame mapped to integers 0-6.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-PP-006
Title	Dataset Split — Proportions Verification
Module	Data Preprocessing
Priority	High
Test Type	Functional
Pre-conditions	Balanced DataFrame available with sufficient samples.
Test Data	Balanced ISEAR DataFrame
Test Steps	1. Run <code>split_dataset(balanced_df)</code> 2. Observe printed split sizes.
Expected Result	Train set is ~70% of total. Val set is ~15%. Test set is ~15%. All sets are non-empty.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-PP-007
Title	Empty String Input Handling
Module	Data Preprocessing
Priority	High
Test Type	Edge Case
Pre-conditions	data_preprocessing.py functions available.
Test Data	Input: "
Test Steps	1. Call clean_text("") 2. Call normalize_text("")
Expected Result	Both functions return an empty string without raising an exception.
Actual Result	
Status	Pass / Fail
Remarks	

4.3 Custom Tokenizer Module

Test Case ID	TC-TK-001
Title	Abbreviation Expansion — Single Abbreviation
Module	Custom Tokenizer
Priority	High
Test Type	Unit
Pre-conditions	EmotionTokenizer class available. DistilBERT tokenizer downloaded.
Test Data	Input: 'omg i am so happy'
Test Steps	1. Initialize EmotionTokenizer() 2. Call expand_abbreviations('omg i am so happy') 3. Observe output.
Expected Result	Output: 'oh my god i am so happy'
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-TK-002
Title	Abbreviation Expansion — Multiple Abbreviations
Module	Custom Tokenizer

Priority	High
Test Type	Unit
Pre-conditions	EmotionTokenizer class available.
Test Data	Input: 'smh omg ngl i kinda feel bad rn'
Test Steps	1. Call expand_abbreviations('smh omg ngl i kinda feel bad rn') 2. Observe output.
Expected Result	All abbreviations expanded: 'shaking my head oh my god not going to lie i kind of feel bad right now'
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-TK-003
Title	Tokenizer Output Shape Verification
Module	Custom Tokenizer
Priority	Medium
Test Type	Functional
Pre-conditions	EmotionTokenizer initialized.
Test Data	Two sample sentences
Test Steps	1. Call encode(['I feel happy today', 'I am so sad']) 2. Check shape of input_ids.
Expected Result	input_ids shape is (2, 128) — batch size 2, max length 128.
Actual Result	
Status	Pass / Fail
Remarks	

4.4 Model Inference Module

Test Case ID	TC-MI-001
Title	Correct Emotion Prediction — Joy
Module	Model Inference
Priority	High
Test Type	Functional
Pre-conditions	Trained model (best_model.pt) and label_map.json exist. EmotionInferencePipeline initialized.

Test Data	Input: 'I just got promoted at work, I am absolutely thrilled!'
Test Steps	1. Call predict('I just got promoted at work, I am absolutely thrilled!') 2. Check final_emotion field.
Expected Result	final_emotion == 'joy'
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-MI-002
Title	Correct Emotion Prediction — Sadness
Module	Model Inference
Priority	High
Test Type	Functional
Pre-conditions	Trained model and pipeline initialized.
Test Data	Input: 'I lost my dog today, he was my best friend for 10 years'
Test Steps	1. Call predict('I lost my dog today, he was my best friend for 10 years') 2. Check final_emotion.
Expected Result	final_emotion == 'sadness'
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-MI-003
Title	Correct Emotion Prediction — Anger
Module	Model Inference
Priority	High
Test Type	Functional
Pre-conditions	Trained model and pipeline initialized.
Test Data	Input: 'I cannot believe they fired me without any warning'
Test Steps	1. Call predict('I cannot believe they fired me without any warning') 2. Check final_emotion.
Expected Result	final_emotion == 'anger'
Actual Result	
Status	Pass / Fail

Remarks	
---------	--

Test Case ID	TC-MI-004
Title	Confidence Score Validity
Module	Model Inference
Priority	High
Test Type	Functional
Pre-conditions	Trained model and pipeline initialized.
Test Data	Input: 'I feel so happy today'
Test Steps	1. Call predict('I feel so happy today') 2. Check all_scores values. 3. Sum all score values.
Expected Result	All score values are between 0.0 and 1.0. Sum of all scores equals approximately 1.0 (within 0.001 tolerance).
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-MI-005
Title	Long Text Truncation Handling
Module	Model Inference
Priority	Medium
Test Type	Edge Case
Pre-conditions	Trained model and pipeline initialized.
Test Data	Input: String repeated 200 times to exceed token limit
Test Steps	1. Generate a string of 1000+ words 2. Call predict(long_text) 3. Observe that no error is raised.
Expected Result	Prediction completes without error. Text is silently truncated to 128 tokens by the tokenizer.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-MI-006
Title	Pure Slang Text Prediction

Module	Model Inference
Priority	Medium
Test Type	Edge Case
Pre-conditions	Trained model and pipeline initialized. Custom tokenizer active.
Test Data	Input: 'smh omg ngl i kinda feel empty inside rn'
Test Steps	1. Call predict('smh omg ngl i kinda feel empty inside rn') 2. Check final_emotion.
Expected Result	After abbreviation expansion, model returns a valid emotion. No error raised. Likely 'sadness' or 'guilt'.
Actual Result	
Status	Pass / Fail
Remarks	

4.5 Sarcasm Detection Module

Test Case ID	TC-SD-001
Title	Sarcasm Detection — Obvious Sarcastic Phrase
Module	Sarcasm Detection
Priority	High
Test Type	Functional
Pre-conditions	SarcasmDetector class available.
Test Data	Input: 'Oh great, another Monday morning'
Test Steps	1. Initialize SarcasmDetector() 2. Call detect('Oh great, another Monday morning') 3. Check is_sarcastic field.
Expected Result	is_sarcastic == True, method == 'rule_based'.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-SD-002
Title	Sarcasm Detection — Positive Word in Negative Context
Module	Sarcasm Detection
Priority	High
Test Type	Functional

Pre-conditions	SarcasmDetector class available.
Test Data	Input: 'Fantastic, my flight got cancelled again'
Test Steps	1. Call detect('Fantastic, my flight got cancelled again') 2. Check is_sarcastic.
Expected Result	is_sarcastic == True (positive word 'fantastic' + negative context 'cancelled').
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-SD-003
Title	Sarcasm Detection — Genuine Positive Sentence
Module	Sarcasm Detection
Priority	High
Test Type	Functional
Pre-conditions	SarcasmDetector class available.
Test Data	Input: 'I am so happy, I got the job!'
Test Steps	1. Call detect('I am so happy, I got the job!') 2. Check is_sarcastic.
Expected Result	is_sarcastic == False.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-SD-004
Title	Emotion Flip on Sarcasm Detection
Module	Sarcasm Detection
Priority	High
Test Type	Integration
Pre-conditions	Trained model and full inference pipeline initialized.
Test Data	Input: 'Oh great, another traffic jam on Monday morning'
Test Steps	1. Call predict('Oh great, another traffic jam on Monday morning') 2. Check raw_emotion and final_emotion. 3. Check adjusted field.
Expected Result	adjusted == True. final_emotion is different from raw_emotion (emotion flipped via EMOTION_FLIP_MAP).
Actual Result	

Status	Pass / Fail
Remarks	

4.6 Negation Handling Module

Test Case ID	TC-NH-001
Title	Double Negation Detection — 'not bad'
Module	Negation Handling
Priority	High
Test Type	Functional
Pre-conditions	NegationHandler class available.
Test Data	Input: 'The result was not bad at all'
Test Steps	1. Initialize NegationHandler() 2. Call resolve('The result was not bad at all') 3. Check has_double_negation and flip_emotion.
Expected Result	has_double_negation == True. flip_emotion == False (double negation = positive, no flip needed).
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-NH-002
Title	Double Negation Detection — 'not unhappy'
Module	Negation Handling
Priority	High
Test Type	Functional
Pre-conditions	NegationHandler class available.
Test Data	Input: 'I was not unhappy with the outcome'
Test Steps	1. Call resolve('I was not unhappy with the outcome') 2. Check has_double_negation.
Expected Result	has_double_negation == True. flip_emotion == False.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-NH-003
Title	Single Negation of Positive Emotion
Module	Negation Handling
Priority	Medium
Test Type	Functional
Pre-conditions	NegationHandler class available.
Test Data	Input: 'I am not happy about this at all'
Test Steps	1. Call resolve('I am not happy about this at all') 2. Check flip_emotion.
Expected Result	flip_emotion == True (negation of positive emotion triggers flip).
Actual Result	
Status	Pass / Fail
Remarks	

4.7 Streamlit Application

Test Case ID	TC-UI-001
Title	Application Launch — No Errors
Module	Streamlit UI
Priority	High
Test Type	Functional
Pre-conditions	step4_app.py present. Models and label_map.json available. Streamlit installed.
Test Data	N/A
Test Steps	1. Run: streamlit run step4_app.py 2. Check terminal for error messages. 3. Check browser opens at localhost:8501.
Expected Result	No error messages in terminal. Browser opens at http://localhost:8501. App displays 4 tabs.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-UI-002
Title	Single Prediction Tab — Correct Emotion Display
Module	Streamlit UI

Priority	High
Test Type	UI Functional
Pre-conditions	App launched at localhost:8501.
Test Data	Input: 'I just got promoted at work!'
Test Steps	1. Navigate to Single Prediction tab 2. Enter: 'I just got promoted at work!' 3. Click 'Detect Emotion' button 4. Observe result.
Expected Result	Emotion displayed as 'JOY' with yellow/gold color. Confidence score shown. Bar chart of all emotion scores displayed.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-UI-003
Title	Single Prediction — Sarcasm Alert Display
Module	Streamlit UI
Priority	High
Test Type	UI Functional
Pre-conditions	App launched. Sarcasm detection enabled.
Test Data	Input: 'Oh great, another Monday morning traffic jam'
Test Steps	1. Enter: 'Oh great, another Monday morning traffic jam' 2. Click Detect Emotion 3. Observe warning message.
Expected Result	Warning message displayed: 'Sarcasm detected — emotion adjusted accordingly'. Adjusted field shows True.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-UI-004
Title	Batch Prediction — CSV Upload and Download
Module	Streamlit UI
Priority	High
Test Type	UI Functional
Pre-conditions	App launched. A CSV file with a 'text' column containing 5 sample sentences prepared.

Test Data	CSV with 5 sample sentences
Test Steps	1. Navigate to Batch Prediction tab 2. Upload CSV file 3. Click 'Run Batch Prediction' 4. Click Download Results button.
Expected Result	Progress bar completes. Results table displays 5 rows with emotion and confidence columns. CSV file downloads successfully.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-UI-005
Title	Prediction Log — Entry Recording
Module	Streamlit UI
Priority	Medium
Test Type	Functional
Pre-conditions	App launched. At least 2 predictions made in Single Prediction tab.
Test Data	Previous single predictions in session
Test Steps	1. Navigate to Prediction Log tab 2. Observe log table.
Expected Result	Log table shows one row per prediction made, with timestamp, text, emotion, confidence, and adjusted columns correctly populated.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-UI-006
Title	Empty Input Handling in UI
Module	Streamlit UI
Priority	High
Test Type	Edge Case
Pre-conditions	App launched.
Test Data	Input: empty string
Test Steps	1. Navigate to Single Prediction tab 2. Leave text area empty 3. Click 'Detect Emotion'
Expected Result	Warning message displayed: 'Please enter some text first.' No crash or error occurs.
Actual Result	

Status	Pass / Fail
Remarks	

Test Case ID	TC-UI-007
Title	Missing Model File Error Handling
Module	Streamlit UI
Priority	High
Test Type	Error Handling
Pre-conditions	step4_app.py present. Model file intentionally removed or path changed to non-existent file.
Test Data	Non-existent model path
Test Steps	1. Set model path to a non-existent file 2. Launch the app 3. Observe error message.
Expected Result	App displays error: 'Model not found at [path]. Train the model first.' App does not crash. st.stop() prevents further execution.
Actual Result	
Status	Pass / Fail
Remarks	

4.8 Logging Module

Test Case ID	TC-LG-001
Title	Prediction Log CSV Creation
Module	Logging
Priority	Medium
Test Type	Functional
Pre-conditions	step4_testing.py present. Trained model available.
Test Data	REAL_WORLD_TEST_CASES from step4_testing.py
Test Steps	1. Run: python step4_testing.py 2. Check logs/ directory after completion.
Expected Result	File logs/predictions.csv created. Contains columns: timestamp, text, true_label, predicted, confidence, correct, adjusted.
Actual Result	
Status	Pass / Fail
Remarks	

Test Case ID	TC-LG-002
Title	Log File Creation with Timestamp
Module	Logging
Priority	Medium
Test Type	Functional
Pre-conditions	step4_testing.py present.
Test Data	N/A
Test Steps	1. Run: python step4_testing.py 2. Check logs/ directory for .log file.
Expected Result	A .log file is created with a timestamp in its filename (e.g., emotion_model_20260326_172000.log). File contains INFO and DEBUG level entries.
Actual Result	
Status	Pass / Fail
Remarks	

5. Test Cases Summary

TC ID	Title	Module	Priority	Type
TC-DC-001	ISEAR Dataset Load — Sample Count	Data Collection	High	Functional
TC-DC-002	ISEAR Dataset Load — Label Distribution	Data Collection	High	Functional
TC-DC-003	ISEAR Dataset Load — Null Labels	Data Collection	High	Functional
TC-PP-001	Text Cleaning — URL Removal	Preprocessing	High	Unit
TC-PP-002	Text Cleaning — HTML Tag Removal	Preprocessing	High	Unit
TC-PP-003	Normalization — Contraction Expansion	Preprocessing	High	Unit
TC-PP-004	Normalization — Slang Expansion	Preprocessing	High	Unit
TC-PP-005	Label Encoding — 7 Emotions Mapped	Preprocessing	High	Functional
TC-PP-006	Dataset Split — Proportions	Preprocessing	High	Functional
TC-PP-007	Empty String Input Handling	Preprocessing	High	Edge Case
TC-TK-001	Abbreviation Expansion — Single	Custom Tokenizer	High	Unit
TC-TK-002	Abbreviation Expansion — Multiple	Custom Tokenizer	High	Unit
TC-TK-003	Tokenizer Output Shape	Custom Tokenizer	Medium	Functional
TC-MI-001	Correct Prediction — Joy	Model Inference	High	Functional
TC-MI-002	Correct Prediction — Sadness	Model Inference	High	Functional
TC-MI-003	Correct Prediction — Anger	Model Inference	High	Functional
TC-MI-004	Confidence Score Validity	Model Inference	High	Functional
TC-MI-005	Long Text Truncation Handling	Model Inference	Medium	Edge Case
TC-MI-006	Pure Slang Text Prediction	Model Inference	Medium	Edge Case
TC-SD-001	Sarcasm Detection — Obvious Phrase	Sarcasm Detection	High	Functional
TC-SD-002	Sarcasm — Positive in Negative Context	Sarcasm Detection	High	Functional
TC-SD-003	Sarcasm — Genuine Positive Sentence	Sarcasm Detection	High	Functional
TC-SD-004	Emotion Flip on Sarcasm	Sarcasm Detection	High	Integration
TC-NH-001	Double Negation — 'not bad'	Negation Handling	High	Functional

TC-NH-002	Double Negation — 'not unhappy'	Negation Handling	High	Functional
TC-NH-003	Single Negation of Positive Emotion	Negation Handling	Medium	Functional
TC-UI-001	App Launch — No Errors	Streamlit UI	High	Functional
TC-UI-002	Single Prediction — Correct Display	Streamlit UI	High	UI Functional
TC-UI-003	Single Prediction — Sarcasm Alert	Streamlit UI	High	UI Functional
TC-UI-004	Batch Prediction — CSV Upload/Download	Streamlit UI	High	UI Functional
TC-UI-005	Prediction Log — Entry Recording	Streamlit UI	Medium	Functional
TC-UI-006	Empty Input Handling in UI	Streamlit UI	High	Edge Case
TC-UI-007	Missing Model File Error Handling	Streamlit UI	High	Error Handling
TC-LG-001	Prediction Log CSV Creation	Logging	Medium	Functional
TC-LG-002	Log File Creation with Timestamp	Logging	Medium	Functional

6. Defect Tracking Template

Any defects identified during test execution should be recorded in the following format:

Field	Description
Defect ID	Unique identifier e.g. BUG-001
Test Case ID	The TC ID that uncovered the defect
Defect Title	Short description of the defect
Severity	Critical / High / Medium / Low
Priority	High / Medium / Low
Steps to Reproduce	Exact steps to reproduce the issue
Expected Result	What should happen
Actual Result	What actually happened
Status	Open / In Progress / Fixed / Closed
Assigned To	Developer responsible for the fix
Date Reported	Date defect was found
Date Fixed	Date defect was resolved
